OdontoSoft — Documento final del proyecto

Sistema de gestión para consultorio odontológico con estructuras de datos

Universidad Autónoma de Occidente (UAO) Proyecto final

Integrante	Código
Clara Mercedes Acevedo Urbano	2240713
Leydi Dayana Narváez	2240494
Jhon Brandon Arboleda Criollo	2243499

1. Introducción

OdontoSoft es una plataforma web para la gestión integral de un consultorio odontológico. Además de las funcionalidades propias de la clínica (autenticación, pacientes, citas e historia clínica), el proyecto tiene como objetivo central **implementar y utilizar cinco estructuras de datos** que resuelven necesidades reales del dominio, almacenando la información en una base de datos **MongoDB**.

El sistema está construido sobre una **arquitectura de tres capas** y desplegado en servidores públicos (Netlify para el frontend y Render para el backend), con la base de datos alojada en MongoDB Atlas.

2. Alcance del sistema

OdontoSoft permite:

- **Autenticación y control de acceso por roles (RBAC):** odontóloga, recepcionista y administrador, con contraseñas cifradas (BCrypt) y sesiones mediante tokens JWT.
- **Gestión de pacientes:** registro, búsqueda y consulta de la ficha del paciente.
- **Agendamiento de citas:** creación con verificación de disponibilidad, cancelación y reprogramación.
- **Historia clínica con odontograma:** registro de valoraciones con odontograma de 32 dientes (notación FDI) y plan de tratamiento.
- **Sala de espera (turnos):** atención de pacientes en orden de llegada — **estructura Cola**.
- **Edición del odontograma con función Deshacer — estructura Pila**.
- **Autocompletado de búsqueda de pacientes — estructura Trie**.
- **Índice de pacientes ordenado por documento — estructura Árbol Binario de Búsqueda**.
- **Ordenamiento del plan de tratamiento según dependencias — estructura Grafo** con orden topológico.

Quedan fuera del alcance la facturación electrónica, la integración con entidades externas (EPS) y la aplicación móvil nativa.

3. Arquitectura y tecnologías

El sistema sigue una arquitectura de **tres capas**:

Capa	Tecnología	Responsabilidad
Presentación	HTML, CSS y JavaScript (sin frameworks)	Interfaz de usuario y visualización de las estructuras
Lógica de negocio	Python + FastAPI (API REST)	Reglas del dominio, seguridad y las estructuras de datos
Datos	MongoDB + GridFS	Persistencia de la información y archivos

Seguridad: autenticación con **JWT**, cifrado de contraseñas con **BCrypt** y **control de acceso por roles (RBAC)**. La comunicación entre capas se realiza mediante peticiones HTTP a una API REST documentada automáticamente (OpenAPI).

4. Modelo de datos (MongoDB) y transacciones

La información se almacena en las siguientes colecciones de MongoDB:

Colección	Contenido
`usuarios`	Cuentas del sistema (nombre, correo, hash de contraseña, rol, estado)
`pacientes`	Fichas de pacientes (documento único, datos personales y de contacto)
`citas`	Agenda (paciente, fecha, hora, estado)
`historias`	Historia clínica: valoraciones, odontograma de trabajo, plan de tratamiento y **pila de deshacer**
`turnos`	Sala de espera (**cola** de atención)
`procedimientos`	Catálogo de procedimientos y sus dependencias (**grafo**)

Las operaciones de escritura (registrar paciente, agendar cita, encolar un turno, aplicar un cambio al odontograma, etc.) son **transacciones** sobre MongoDB que insertan o actualizan documentos de forma atómica, validando previamente las reglas del negocio (por ejemplo, documento o correo únicos, u horario disponible).

5. Estructuras de datos implementadas

Se implementaron **cinco estructuras de datos**, respetando la distribución exigida: **dos lineales** (Cola y Pila), **dos jerárquicas** (Árbol Binario de Búsqueda y Trie) y **un grafo**. Todas se programaron **desde cero** (sin usar librerías de estructuras) en el paquete `backend/estructuras/`.

En el producto, cada estructura está **integrada de forma transparente** en el módulo donde tiene sentido clínico (no en una pantalla técnica aparte). Para la evaluación, la aplicación incluye además una página de **anexo técnico** (`estructuras.html`, sin enlace en el menú) que reúne las cinco en un solo lugar.

5.1 Cola (FIFO) — Sala de espera

- **Archivo:** `backend/estructuras/cola.py`
- **Qué es:** estructura lineal donde el primer elemento en entrar es el primero en salir (*First In, First Out*).
- **Uso en OdontoSoft:** la sala de espera. Los pacientes se atienden en el orden en que llegan; "llamar al siguiente" extrae al primero de la cola.
- **Por qué esta estructura:** la atención por estricto orden de llegada es, por definición, un comportamiento FIFO. La Cola lo modela de forma natural y justa.
- **Implementación:** lista enlazada simple con punteros al frente y al final, de modo que encolar y desencolar no requieren desplazar elementos.
- **Operaciones y complejidad:**

Operación	Complejidad
`encolar` (enqueue)	O(1)
`desencolar` (dequeue)	O(1)
`frente` (peek)	O(1)

- **Dónde se ve (en el producto):** módulo **Sala de espera** → los pacientes se registran al llegar y "Llamar siguiente" aplica `desencolar()` (FIFO).

5.2 Pila (LIFO) — Deshacer odontograma

- **Archivo:** `backend/estructuras/pila.py`
- **Qué es:** estructura lineal donde el último elemento en entrar es el primero en salir (*Last In, First Out*).
- **Uso en OdontoSoft:** función **Deshacer** sobre el odontograma. Cada cambio de estado de un diente se apila; al deshacer se retira el último cambio y el diente vuelve a su estado anterior.
- **Por qué esta estructura:** "deshacer" siempre revierte la acción **más reciente** primero, que es exactamente el comportamiento LIFO de una pila.

- **Implementación:** lista enlazada simple con puntero a la cima. La pila se persiste en el documento de la historia clínica (campo `pila_deshacer`).
- **Operaciones y complejidad:**

Operación	Complejidad
`apilar` (push)	O(1)
`desapilar` (pop)	O(1)
`cima` (peek)	O(1)

- **Dónde se ve (en el producto):** módulo **Historia clínica** → odontograma: cada cambio en un diente se apila y el botón "Deshacer" lo revierte (desapilar).

5.3 Árbol Binario de Búsqueda (BST) — Pacientes por documento

- **Archivo:** `backend/estructuras/arbol.py`
- **Qué es:** árbol binario en el que, para cada nodo, las claves menores quedan a la izquierda y las mayores a la derecha.
- **Uso en OdontoSoft:** índice de pacientes ordenados por número de documento. Permite buscar un paciente eficientemente y listar a todos en orden ascendente.
- **Por qué esta estructura:** sobre un conjunto ordenable (los documentos), un BST ofrece **búsqueda O(log n)** (frente a O(n) de una búsqueda lineal) y su **recorrido inorden** entrega los elementos ya ordenados sin un paso adicional de ordenamiento.
- **Operaciones y complejidad** (h = altura; h ≈ log n si está equilibrado):

Operación	Complejidad
`insertar`	O(h)
`buscar`	O(h)
`inorden` (listado ordenado)	O(n)

- **Complemento — Árbol del odontograma:** el mismo archivo incluye `ArbolOdontograma`, un árbol jerárquico de tres niveles (boca → cuadrante → diente) que modela que la dentadura es, por naturaleza, una estructura de árbol y permite obtener resúmenes por cuadrante.
- **Dónde se ve (en el producto):** módulo **Pacientes** → opción "Ordenar: documento" (recorrido inorden del BST). El anexo técnico muestra además la forma del árbol.

5.4 Trie (árbol de prefijos) — Autocompletado

- **Archivo:** `backend/estructuras/trie.py`
- **Qué es:** árbol donde cada camino desde la raíz representa un prefijo; las claves que comparten prefijo comparten camino.

- **Uso en OdontoSoft:** autocompletado de la búsqueda de pacientes. Al escribir, el sistema sugiere los pacientes cuyo nombre, apellido o documento comienza con el texto ingresado.
- **Por qué esta estructura:** la búsqueda por prefijo en un Trie depende únicamente de la **longitud del texto escrito (O(L))**, no de la cantidad total de pacientes. Es la estructura idónea para sugerencias en tiempo real (typeahead).
- **Operaciones y complejidad** (L = longitud de la clave, k = nº de coincidencias):

Operación	Complejidad
`insertar`	O(L)
`buscar`	O(L)
`autocompletar`	O(L + k)

- **Dónde se ve (en el producto):** módulo **Pacientes** (y búsqueda de Historia) → al escribir aparece el autocompletado de pacientes; tolera tildes y mayúsculas.

5.5 Grafo dirigido — Dependencias de procedimientos

- **Archivo:** `backend/estructuras/grafo.py`
- **Qué es:** conjunto de nodos conectados por aristas dirigidas. Aquí, un arco A → B significa "el procedimiento A debe realizarse antes que B".
- **Uso en OdontoSoft:** modela las **precedencias entre procedimientos** del plan de tratamiento (por ejemplo, *Endodoncia* → *Corona*, *Profilaxis* → *Blanqueamiento*). A partir del grafo se calcula un **orden topológico**: una secuencia válida para ejecutar los procedimientos respetando todas las dependencias. Si existe un ciclo (dependencias contradictorias), el plan se marca como inconsistente.
- **Por qué esta estructura:** las relaciones de precedencia "esto antes que aquello" forman un grafo dirigido; el **orden topológico** es precisamente el algoritmo que resuelve en qué orden ejecutar tareas con dependencias.
- **Implementación:** lista de adyacencia; orden topológico mediante el **algoritmo de Kahn** (basado en grados de entrada), que además detecta ciclos.
- **Operaciones y complejidad** (V = nodos, E = aristas):

Operación	Complejidad
`agregar_nodo` / `agregar_arista`	O(1)
`orden_topologico` (Kahn)	O(V + E)
`bfs` / `dfs`	O(V + E)

- **Dónde se ve (en el producto):** módulo **Historia clínica** → "Secuencia recomendada de procedimientos": al seleccionar procedimientos, el sistema añade los prerrequisitos y calcula el orden de ejecución (orden topológico). El anexo técnico dibuja el grafo completo.

Resumen de cumplimiento

Grupo exigido	Estructuras implementadas
2 de {Lista, Pila, Cola}	**Cola** y **Pila**
2 de {Árbol, Trie, Heap}	**Árbol Binario de Búsqueda** y **Trie**
1 Grafo	**Grafo dirigido** (orden topológico)

6. Despliegue

- **Frontend:** Netlify (contenido estático de `frontend/`).
- **Backend:** Render (servicio web con FastAPI/Uvicorn).
- **Base de datos:** MongoDB Atlas (clúster gratuito M0).

El detalle paso a paso está en `docs/DESPLIEGUE.md`. Los enlaces públicos se publican en el `README.md` del repositorio.

7. Trabajo colaborativo (Git)

El desarrollo se organizó en **tres ramas**, una por integrante, integradas sobre `main` mediante *pull requests*:

Integrante	Rama	Aporte
Jhon Brandon Arboleda Criollo	`jhon`	Árbol/BST, Trie y Grafo (backend) + integración con MongoDB
Leydi Dayana Narváez	`leydi`	Cola y Pila (backend), documentación y despliegue
Clara Mercedes Acevedo Urbano	`clara`	Frontend visual de las estructuras y pruebas

El flujo completo (ramas, commits y merge) está en `docs/GUIA_GIT.md`.

8. Conclusiones

- Se implementaron las cinco estructuras de datos exigidas, cada una resolviendo un problema **real y pertinente** del dominio odontológico, y no como un ejercicio aislado.
- La elección de cada estructura se justifica por su complejidad y por el comportamiento que modela (FIFO, LIFO, búsqueda ordenada, búsqueda por prefijo y precedencias).
- El sistema integra las estructuras con una base de datos **MongoDB** real, una **interfaz visual** que permite observarlas en funcionamiento y un **despliegue público**, cumpliendo con todos los productos solicitados en la rúbrica.